CSS進階

NTNU CSIE CAMP

講師：在哭

複習一下
前面學了什麼...

```html
<ul>
    <li>這是第一點</li>
    <li>這是第二點</li>
    <li>這是第三點</li>
</ul>

<ol>
    <li>這是第一點</li>
    <li>這是第二點</li>
    <li>這是第三點</li>
</ol>
```

```css
h1 {}      /* 元素選擇器 */
.類別 {}   /* 類別選擇器 */
#id {}     /* id選擇器   */
* {}       /* 通用選擇器 */
```

```html
<h1 id = "id-1"> 帶有id屬性的h1標題 </h1>
<h1 class = "class-1"> 帶有class屬性的h1標題 </h1>
```

複習一下
前面學了什麼...

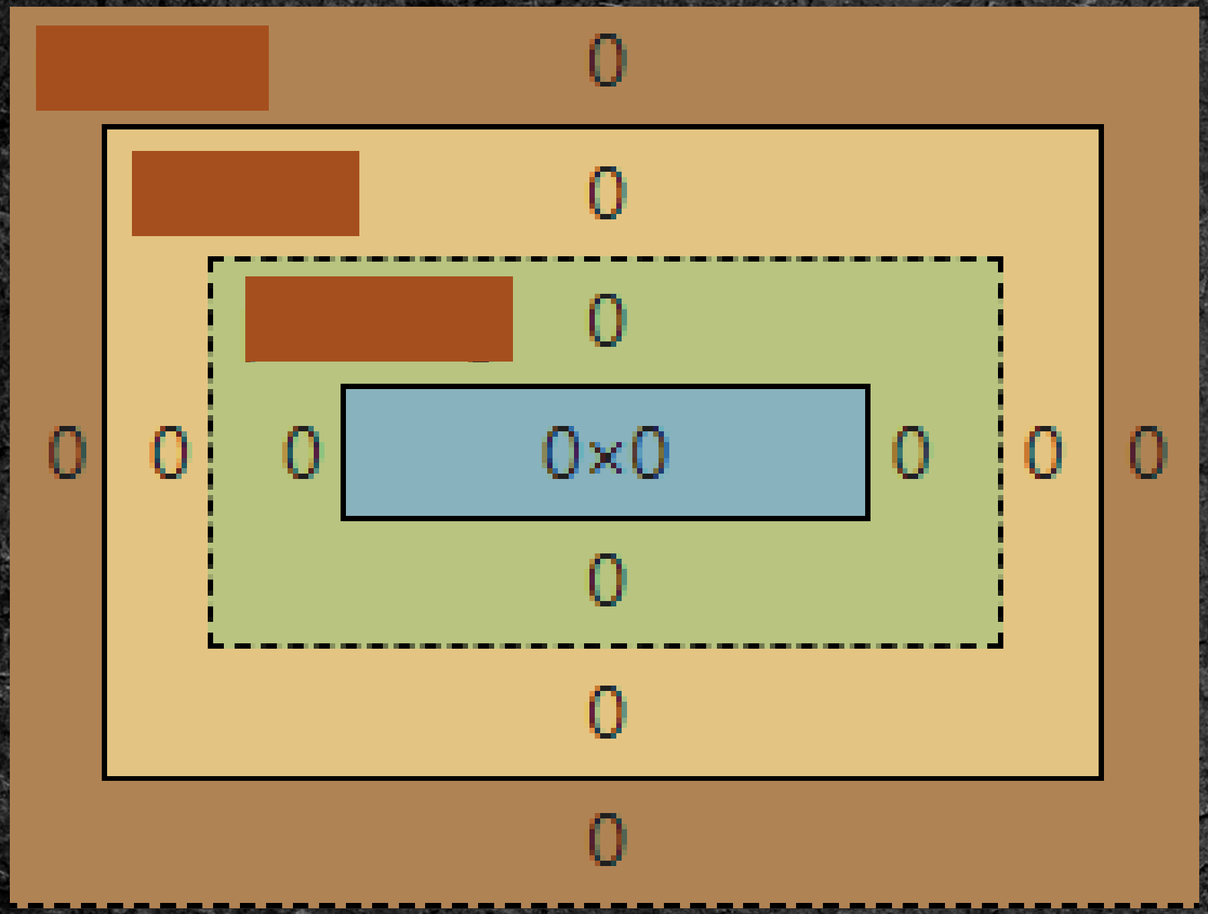
```html
<ul>
    <li>這是第一點</li>
    <li>這是第二點</li>
    <li>這是第三點</li>
</ul>

<ol>
    <li>這是第一點</li>
    <li>這是第二點</li>
    <li>這是第三點</li>
</ol>
```

- 這是ul
- 這是ul

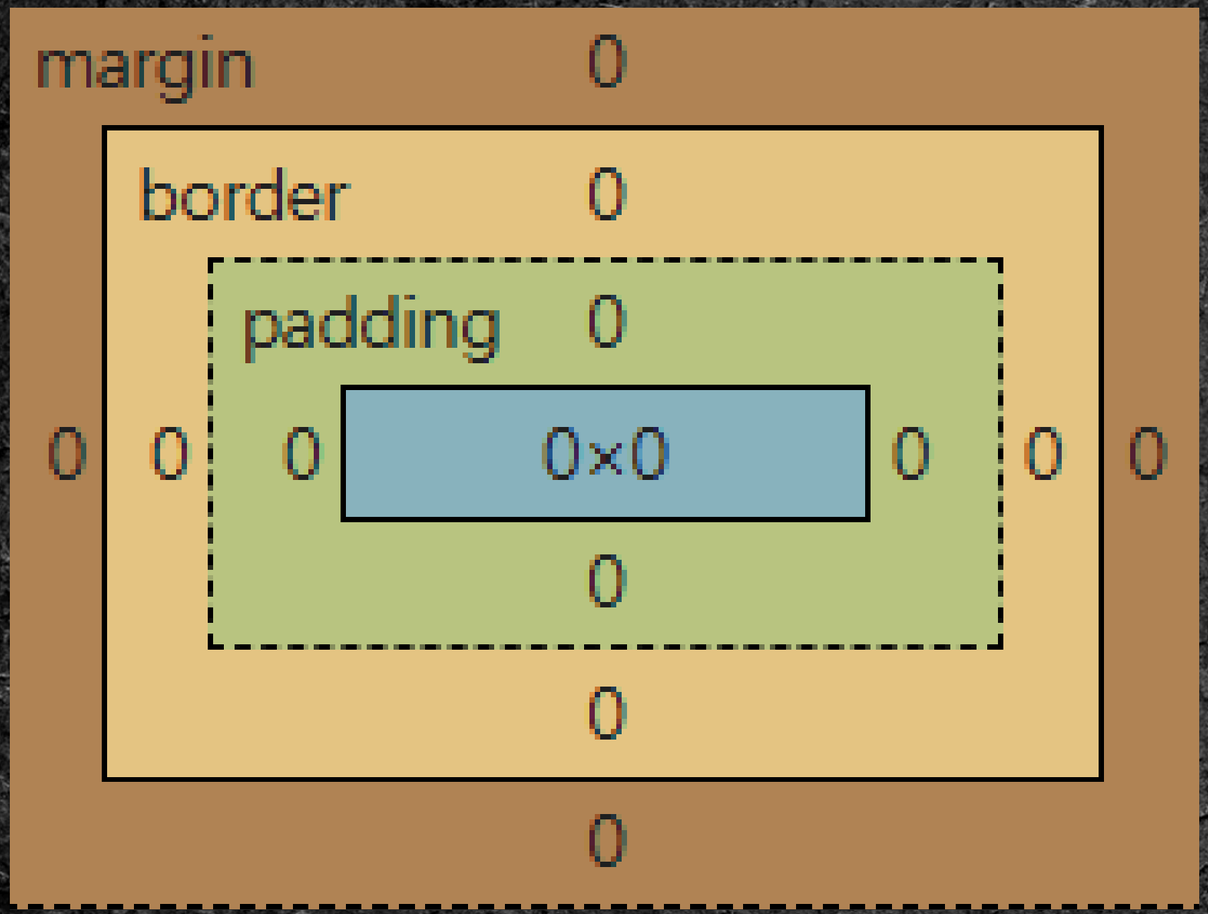
1. 這是ol
2. 這是ol

```css
h1 {}       /* 元素選擇器 */
.類別 {}    /* 類別選擇器 */
#id {}      /* id選擇器   */
* {}        /* 通用選擇器 */
```

```html
<h1 id = "id-1"> 帶有id屬性的h1標題 </h1>
<h1 class = "class-1"> 帶有class屬性的h1標題 </h1>
```



- 藍色： 元素大小
- 綠色： 內邊框大小
- 黃色： 邊框大小
- 棕色： 外邊框大小

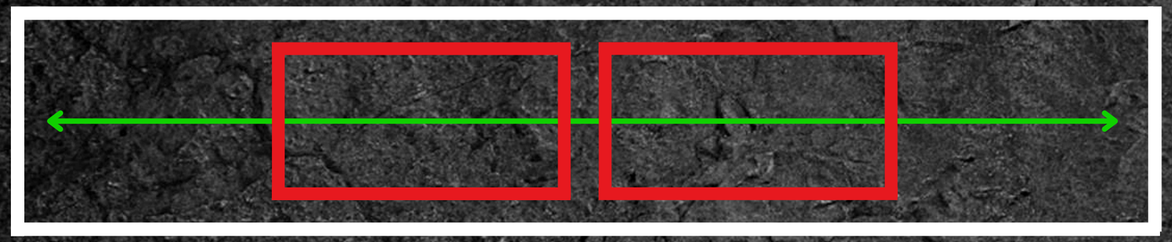


- 藍色： 元素大小
- 綠色： 內邊框大小
- 黃色： 邊框大小
- 棕色： 外邊框大小

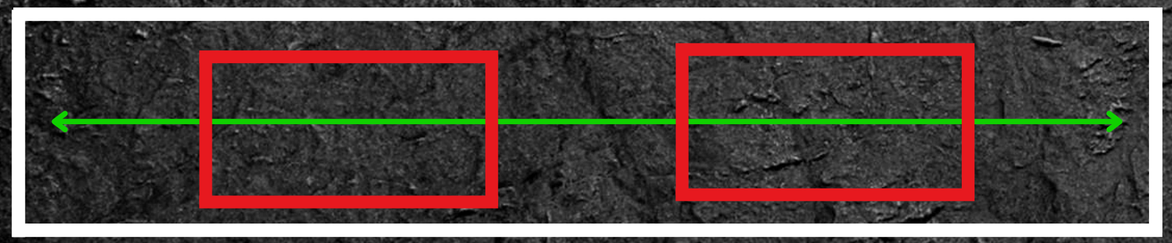
flex 布局

justify-content

決定元素在主軸的對齊方式



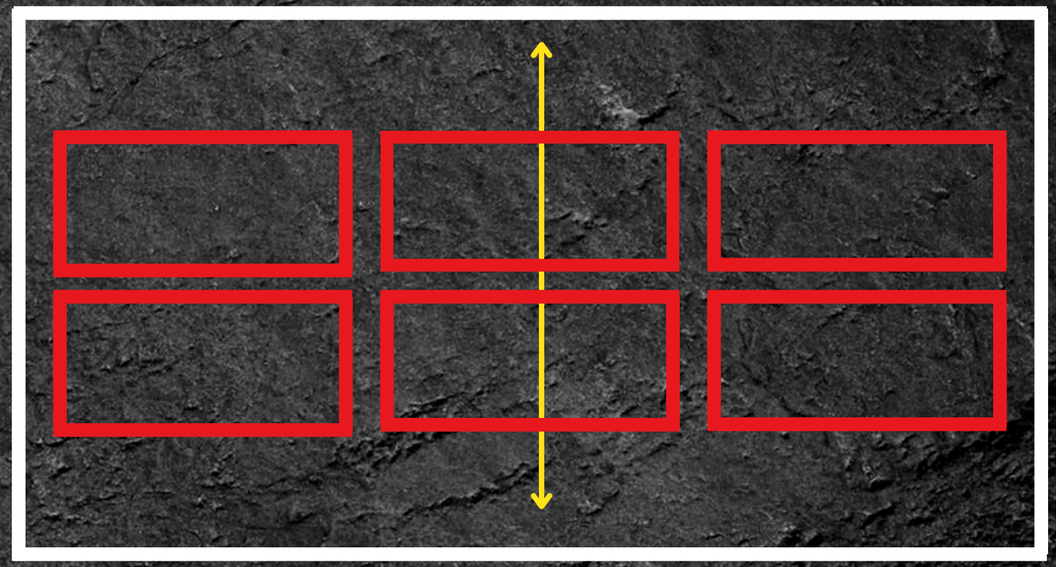
justify-item: center



justify-item: space-around

align-content

決定元素在交錯軸的多行對齊方式



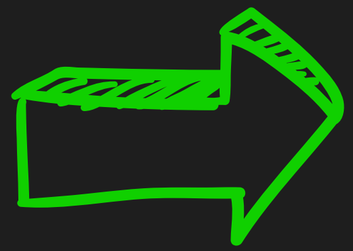
align-content: center

小測驗

```
header
{
    background-color: #eec540;
    height: 100px;
    width: 100%;
    display: flex;
    position: relative;
}
```

小測驗

```
header
{
    background-color: #eec540;
    height: 100px;
    width: 100%;
    display: flex;
    position: relative;
}
```



背景顏色
高度
寬度
顯示方法
位置

那麼CSS到底能
做到什麼程度呢?

接下來的網頁效果...

完全沒有JavaScript!

https://zaiku567.github.io/2026CSIEcamp/

Slido

課堂中有任何問題歡迎先在上面提問

實作環節時會一次解答

Task00
基本準備

```
*
{
    border: 0px;
    padding: 0px;
    margin: 0px;
}
```

每個網頁都有自己的預設排版...

因此在最開始先清除預設避免干擾

Task01

字體效果

~~張家兄弟滑起來~~

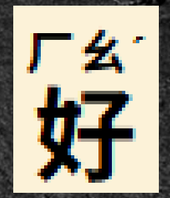
ㄏㄠˇ 好 那今天呢

統神

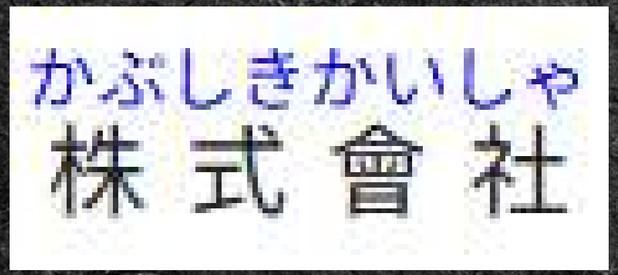
早餐吃到飽

1-1. <ruby>

基本： `<ruby>好<rt>ㄏㄠˇ</rt></ruby>`



若瀏覽器不支援<ruby>：



JCinfo.net

```
<ruby>株式<rp>(</rp><rt>かぶしき</rt><rp>)</rp></ruby>
```

1-2. 長度單位

1px	1像素
100%	父元素的100%
100vw	視窗的100%
1em	父元素的字體大小的1倍
1rem	瀏覽器的預設字體大小的1倍

1-3. 自訂屬性

特殊字體標籤

標籤名		
預設樣式	斜體	粗體

（有強調語意）

<i>	<b>
斜體	粗體

以上效果只是「預設」
代表也可以自己改

```
<em class="slogan">早餐吃到飽</em>
```

早餐吃到飽

以上效果只是「預設」
代表也可以自己改

```
<em class="slogan">早餐吃到飽</em>
```

```
.slogan
{
    font-size: 5vw;
    font-style: normal;
    font-weight: bolder;
    background: linear-gradient(transparent 60%, #eec540 60%);
}
```

早餐吃到飽

以上效果只是「預設」
代表也可以自己改

```
<em class="slogan">早餐吃到飽</em>
```

```
.slogan
{
    font-size: 5vw;
    font-style: normal;
    font-weight: bolder;
    background: linear-gradient(transparent 60%, #eec540 60%);
}
```

ViewportWidth
視窗寬度

早餐吃到飽

vw/vh vs. %

ViewportWidth
ViewportHeight

中間沒有點

ViewportWidth 視窗寬度
ViewportHeight 視窗高度

視窗的?%

父元素的?%

```
<div class="文字">
    <p>
        <span class="加大"><ruby>好<rt>ㄏㄠˇ</rt>
        <br>:沒有沒有 我都帶好幾份去吃 <br>那是你嘛
    </p>
    <h2><em class="slogan">早餐吃到飽</em></h2>
</div>
```

父元素

子元素

Memento Mori

width: 100%

張家兄弟滑起來 統神 國動

我的美食地圖

父元素寬度

被右方滾動軸縮短

ㄏㄠˇ 那今天呢風光明媚風和日麗 因為我以前在唸書的時候 我常常覺得很奇怪就是我到學校然後看他們吃什麼早餐都是吃一份蛋餅配一杯奶茶 或是 一個漢堡配一杯奶茶 或是一份蘿蔔糕配一杯奶茶 每次吃完都說啊我吃飽了我心裡就想說這真的可以吃飽?因為你知道我通常早餐我都點3份可能 3個我想說這樣就能吃飽？一定是唬人的嘛 我才想到說大家都是要面子的

:沒有沒有 我都帶好幾份去吃

那是你嘛 你不要臉嘛 我說正常的 我就想說 不行 我們一定要做個企劃 就是有一天要讓自己的胃滿足大滿足 所以今天的企劃就是

width: 100vw

看回憶

視窗寬度

招牌蹦蛙跳

輸入「招牌蹦蛙跳」並按下 ENTER 吃早餐(備份)

1-4. font- 文字屬性

font-size 文字大小

font-weight 文字寬度

1-5. text- 文字呈現

text-decoration: line-through; underline; 文字裝飾

~~張家兄弟滑起來~~

統神

「:hover」：當鼠標在元素上時

「:hover」：當鼠標在元素上時

```css
header a:hover{
    text-decoration: underline;
    cursor: pointer;
}
```

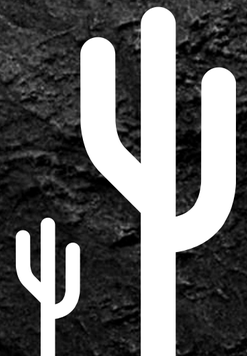
「:hover」：當鼠標在元素上時

```
header a:hover{
    text-decoration: underline;
    cursor: pointer;
}
```

更改鼠標成pointer

實作時間

10分鐘



重點整理:

Task01

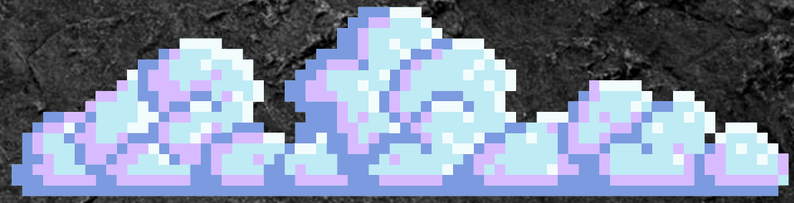
- ▶ <ruby>👍<rt>ㄅㄧㄠˋ</rt></ruby>
- ▶ width: 100%; / 100vw; font-size: 1px; / 1em; / 1rem;
- ▶ text-decoration: underline; / line-through;
- ▶ 元素 :hover{}
- ★ background: linear-gradient (transparent 60%, 顏色);

~~張家兄弟滑起來~~

統神

早餐吃到飽

Task02
選單效果



我的美食地圖

我的美食地圖

看地圖

我的收藏

2-1. position 屬性

position: relative

相對原本的位置移動

2-1. position 屬性

整個區域被相對原本位置
往下推50px

position: relative
相對原本的位置移動

position: absolute

依據父元素的位置放置

不再佔據排版位置

因此適合用在元素會互相覆蓋時

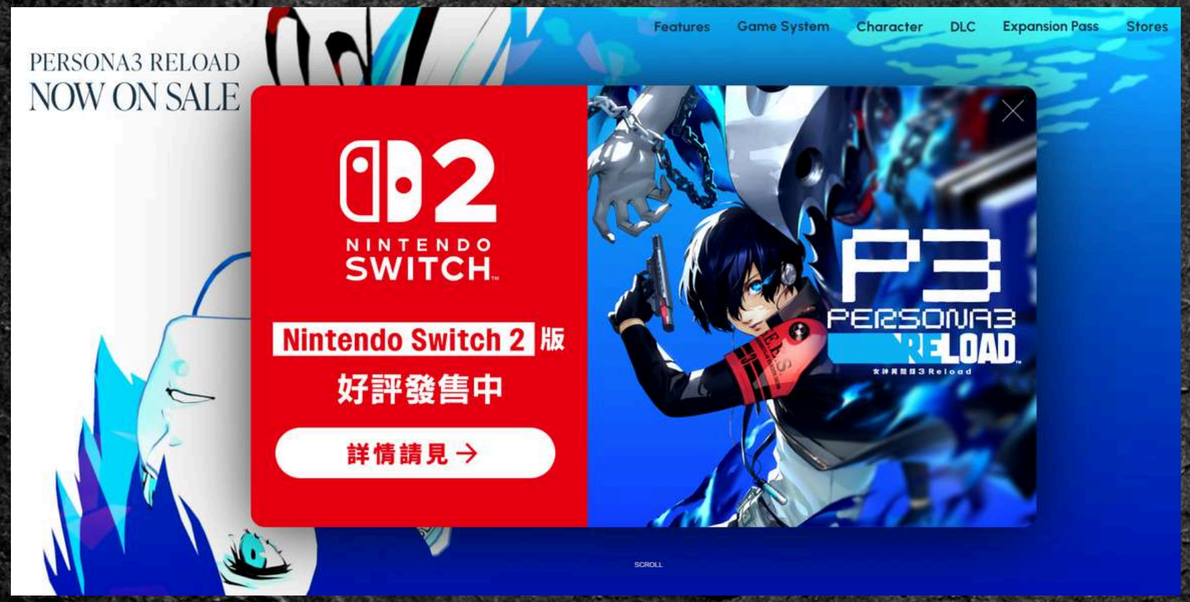
te

也就是說可以這樣直接蓋上去

蓋時

position: fixed 依據視窗放置

最適合用來做彈出視窗



廣告

互動視窗

position: sticky

佔排版空間&可指定要黏視窗的哪邊

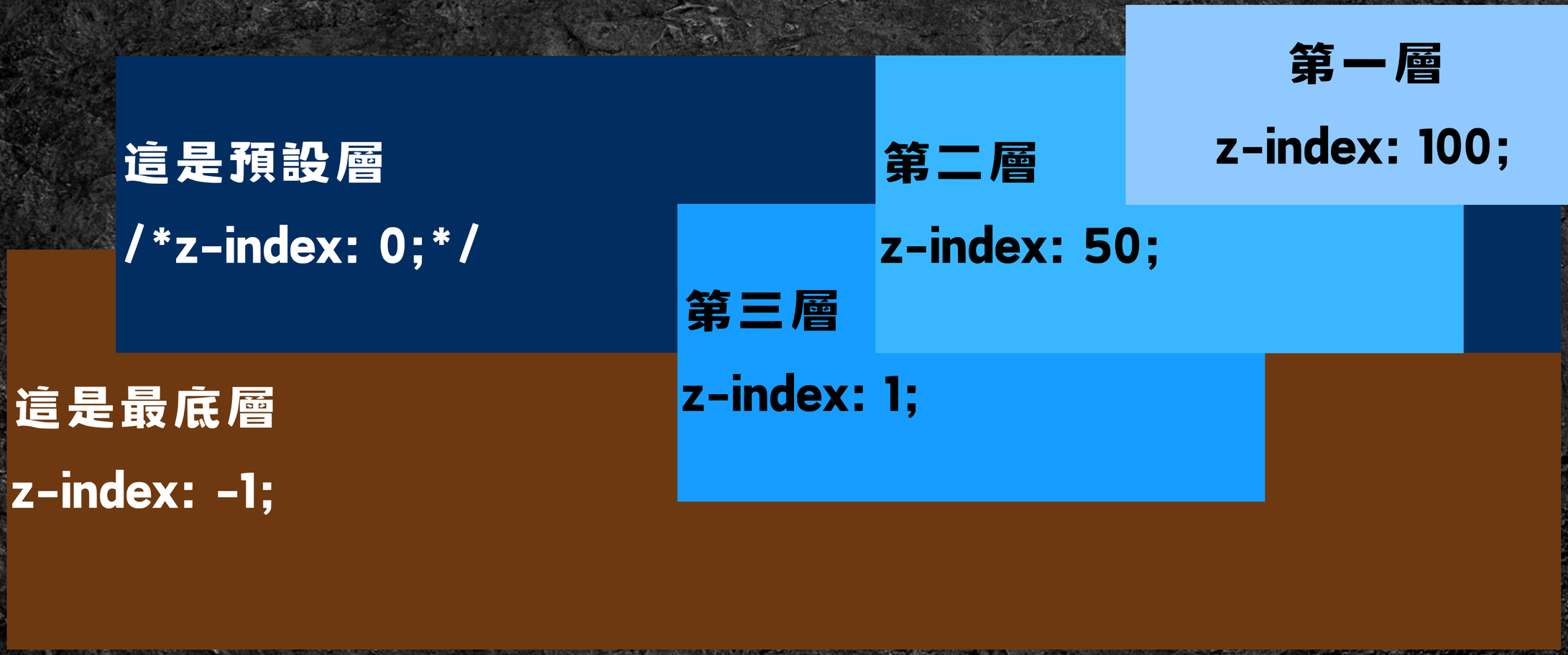
position: sticky

佔排版空間&可指定要黏視窗的哪邊

```
position: sticky;
top:0;
height: 50px;
width: 100%;
background-color: rgb(4, 71, 49);
z-index: 100;
```

值越高元素越上面

z-index


這是預設層
/*z-index: 0;*/
第一層
z-index: 100;
第二層
z-index: 50;
第三層
z-index: 1;
這是最底層
z-index: -1;

z-index

z-index

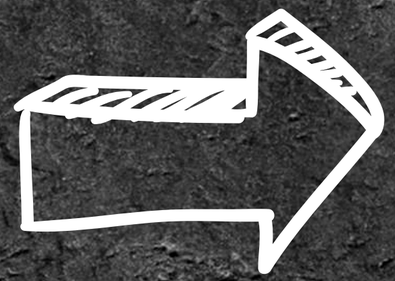
z-index 練習

2-2. <details><summary>

```
<nav>
    <details class="menu">
        <summary>我的美食地圖</summary>
        <ul class="submenu">
            <li><a href="/map/map.html">看地圖</a></li>
            <li><a href="/saved/saved.html">我的收藏</a></li>
        </ul>
    </details>
</nav>
```

<summary>

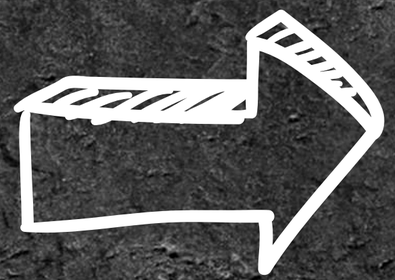
其他內容



<details> (menu)

<summary>

其他內容



<summary>

 (submenu)

<a>

<a>

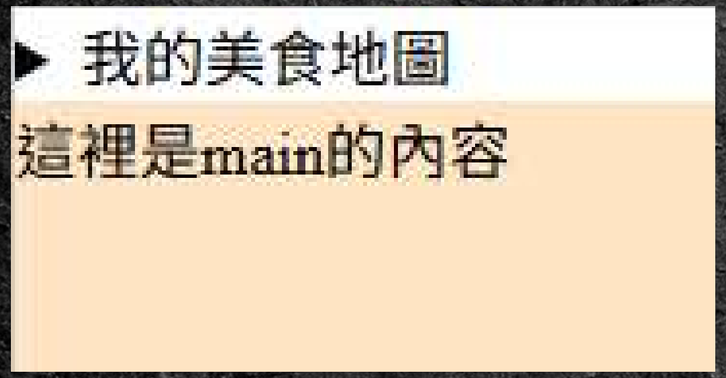
其他內容

<details> (menu)

預設上來說...

submenu未展開時是display:none

展開後是display:block



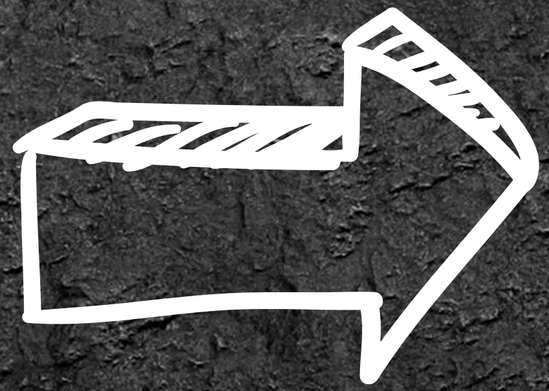
預設上來說...

submenu未展開時是display:none

展開後是display:block

▶ 我的美食地圖

這裡是main的內容



▼ 我的美食地圖

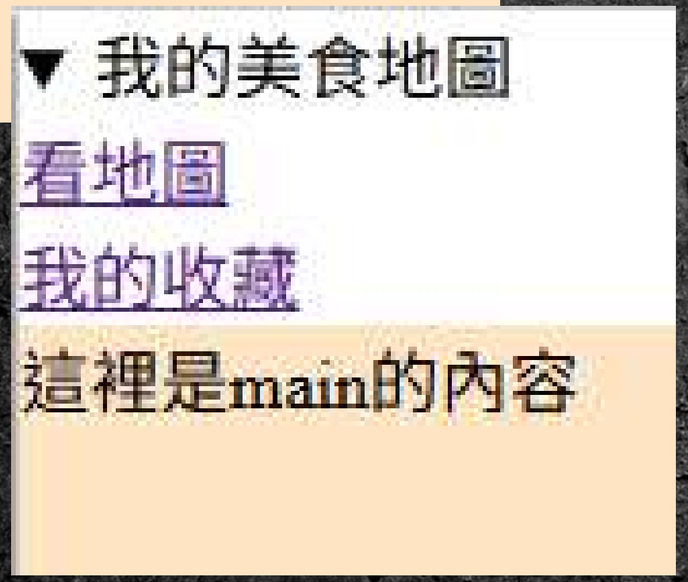
看地圖

我的收藏

這裡是main的內容

會列入排版並將版面下推

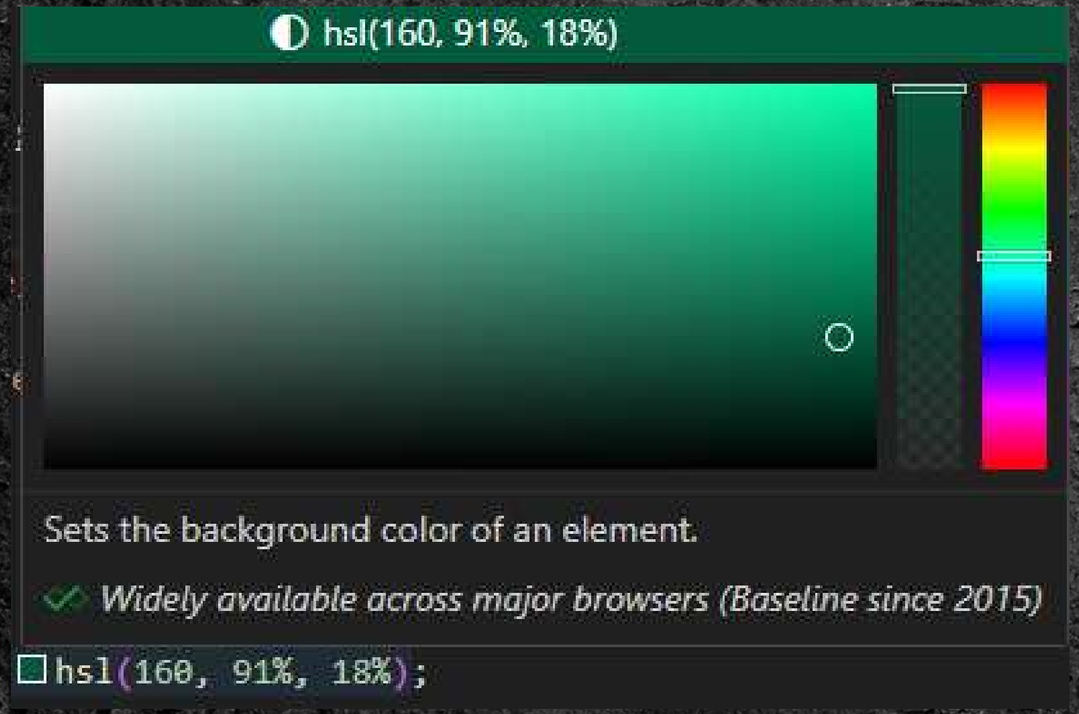
2-3. 用<details>做導覽列



2-3. 用<details>做導覽列

• 背景顏色與文字置中

先挑個喜歡的顏色區分不同區域



▼ 我的美食地圖

看地圖

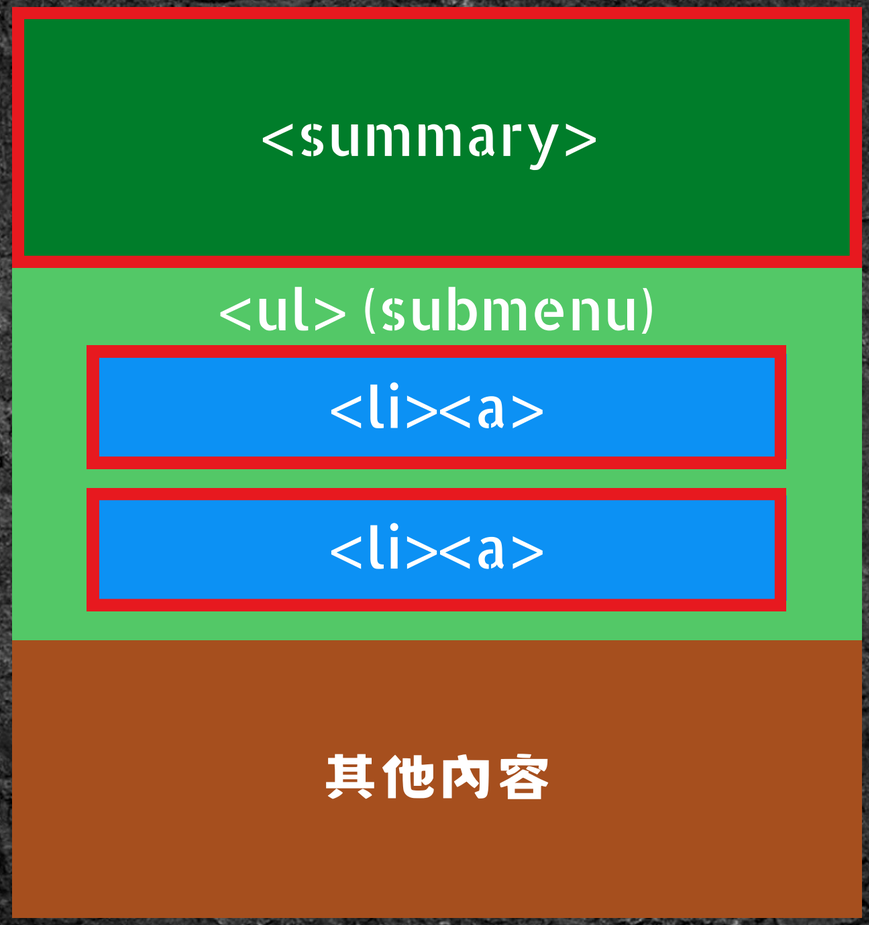
我的收藏

這裡是main的內容

• 背景顏色與文字置中

接著把「裝有文字」的元素加上flex

```
.menu summary{
  display: flex;
  justify-content: center;
  /*align-items: center;*/
}
.submenu li a{
  display: flex;
  justify-content: center;
  /*align-items: center;*/
  text-decoration: none;
```


<summary>
 (submenu)
<a>
<a>
其他內容

• 背景顏色與文字置中

如果不寫在summary而是.menu ...

<summary>

 (submenu)

<a>

<a>

其他內容

• 背景顏色與文字置中

如果不寫在summary而是.menu ...

<summary>

 (submenu)

<a>

<a>

其他內容

呈現flex的是<summary>和.submenu

▼我的美食地圖 看地圖

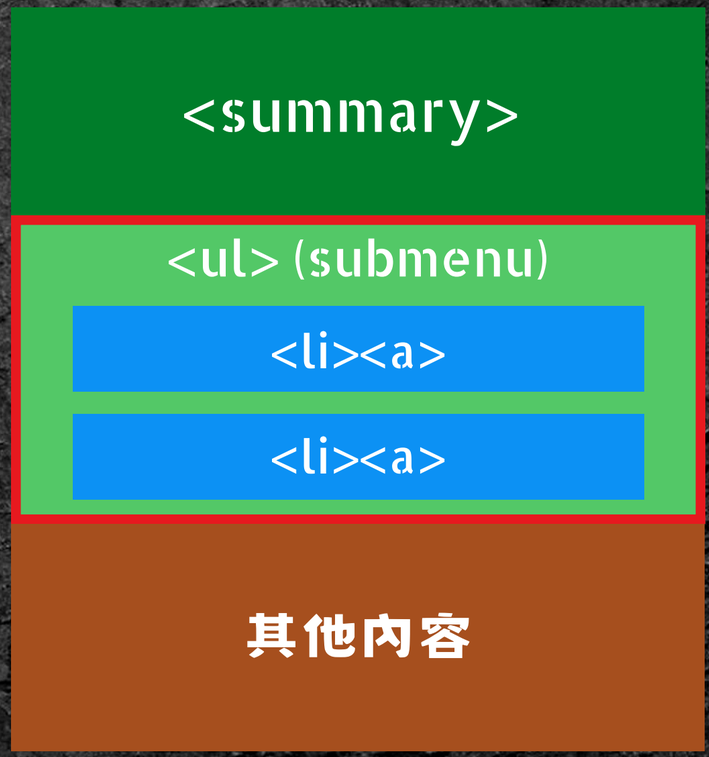
• 我的收藏

這裡是main的內容

<nav>裡的文字若為flex則list-style預設消失

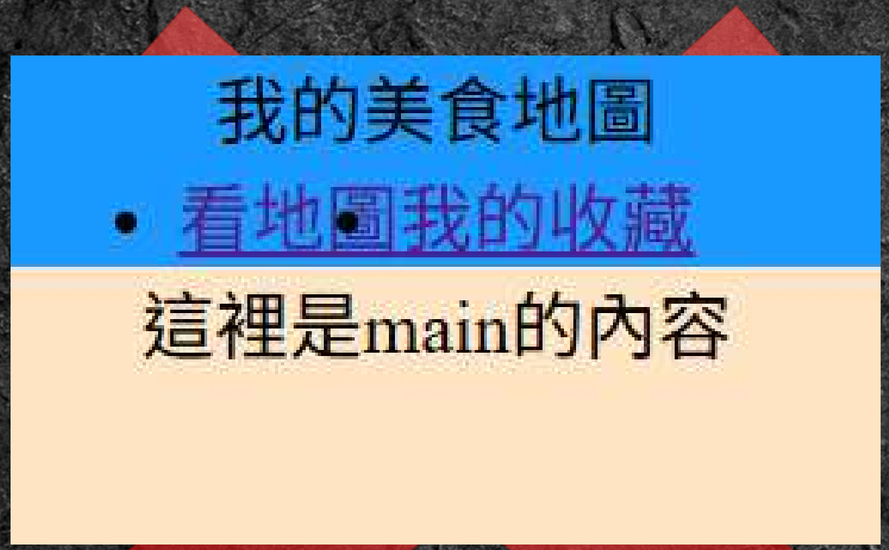
• 背景顏色與文字置中

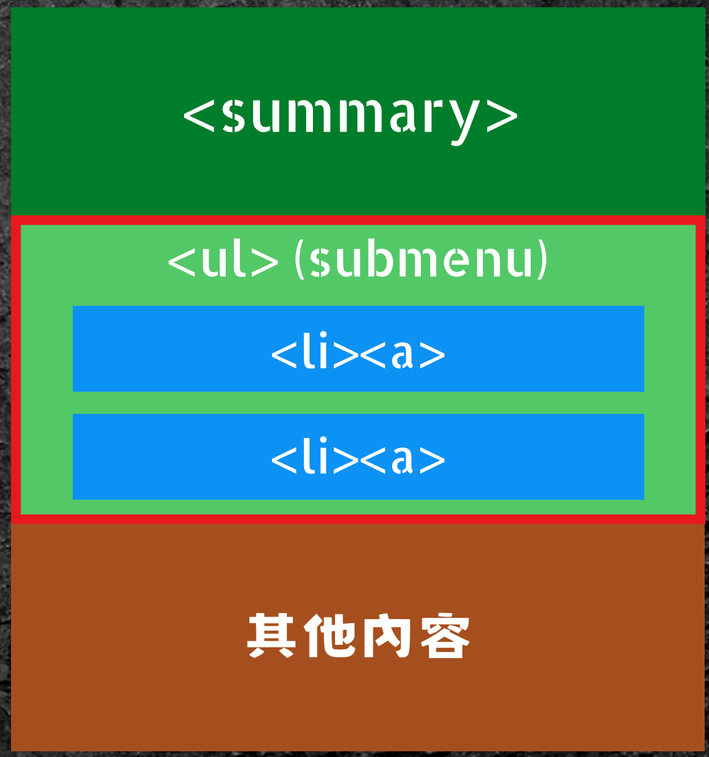
如果不寫在.submenu的子元素<a>而是.submenu ...



• 背景顏色與文字置中

如果不寫在.submenu的子元素<a>而是.submenu ...



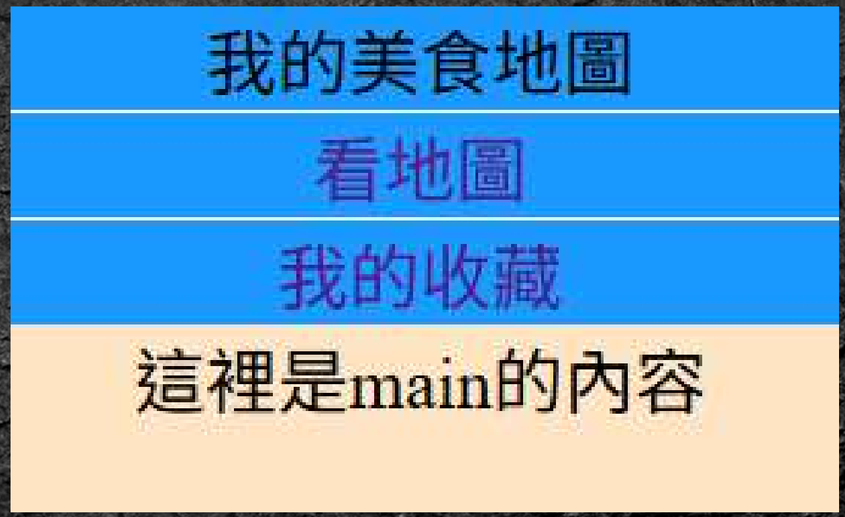


呈現flex的是.submenu下的兩個

• 加邊界線與padding

由於我們想要每個選項一個格子

因此加入邊界的元素是<a>

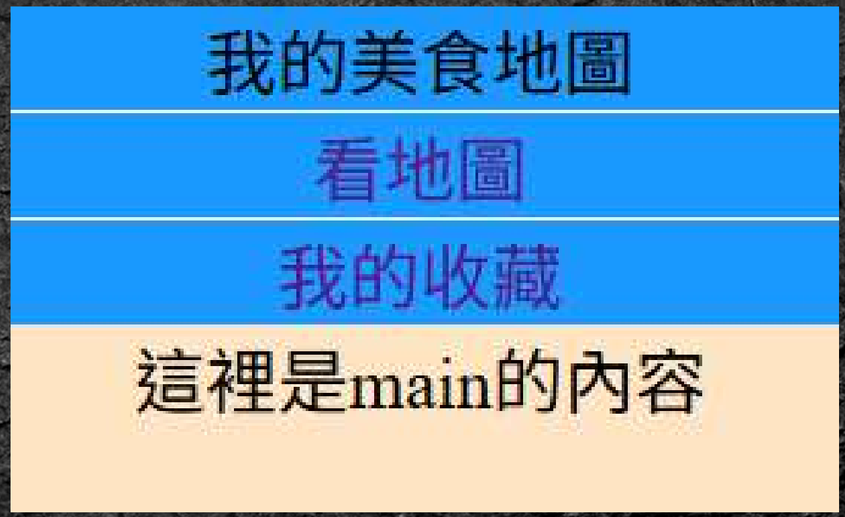


• 加邊界線與padding

由於我們想要每個選項一個格子

因此加入邊界的元素是<a>

```
.submenu li a{
  border-top:1px solid white;
  display: flex;
  justify-content: center;
  /*align-items: center;*/
  text-decoration: none;
}
```



• 加邊界線與padding

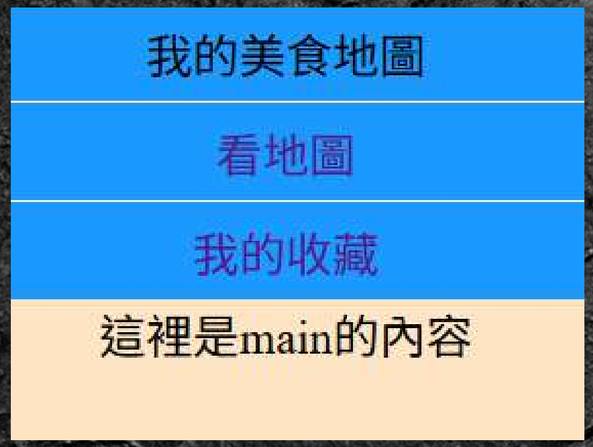
現在格子裡的字看起來隔得太近

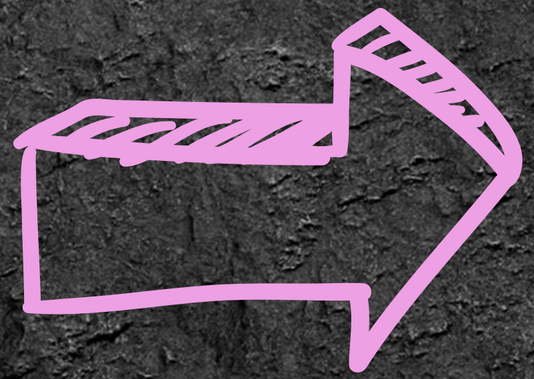
為文字都加上內邊距看起來更自然

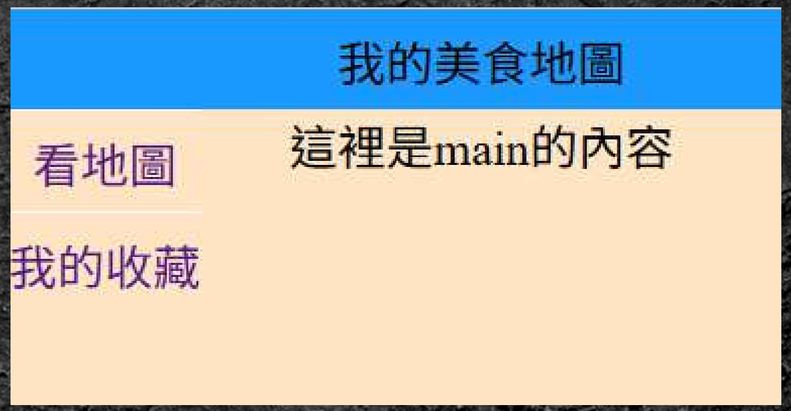
- 把.submenu改成非文檔流

現在的submenu會被列入排版

將.submenu的position改成absolute

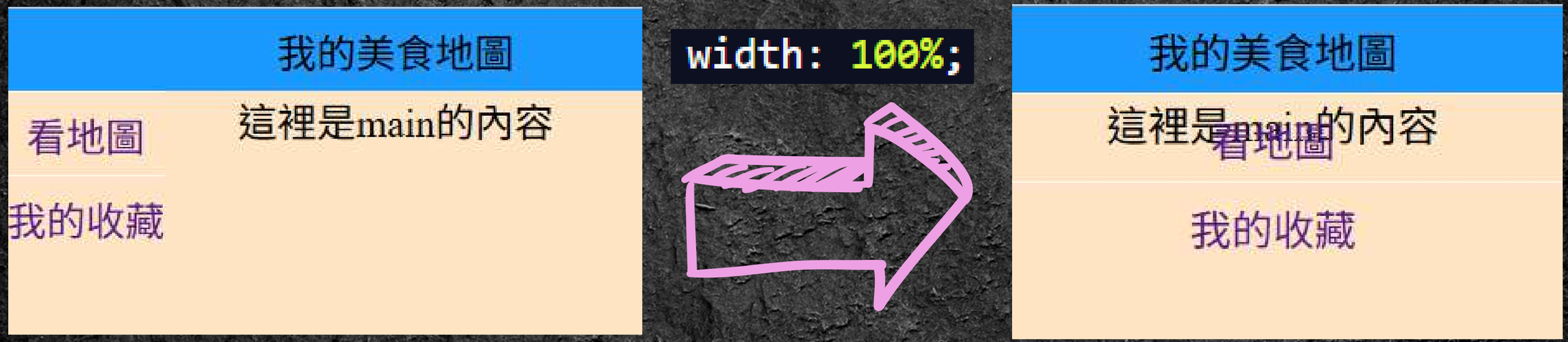




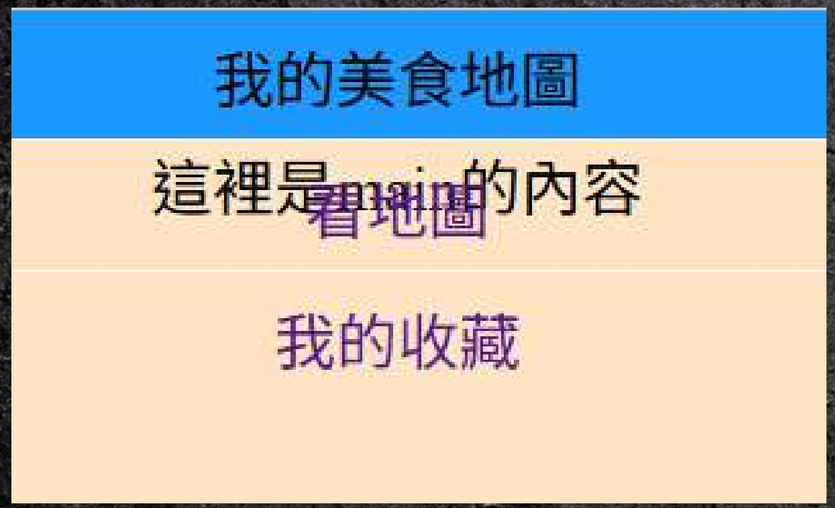


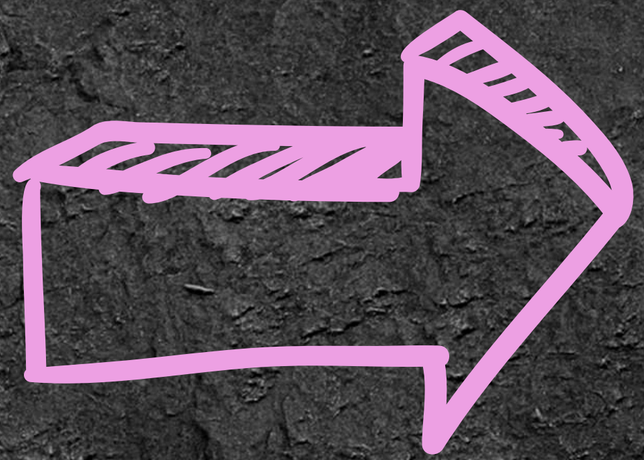

```
position: absolute;
```

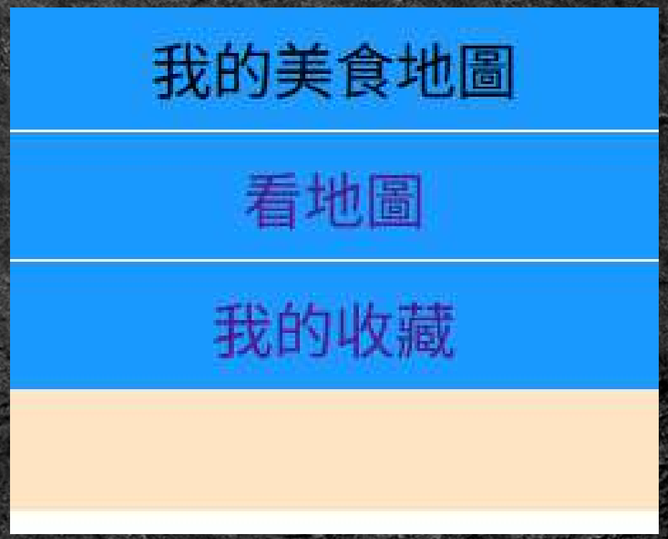
寬度因為不跟父元素了所以預設為內容寬度



加個背景顏色把後面蓋住








```
background-color:rgb(28, 153, 326);
```

大功告成


我的美食地圖
這裡是
main的內容
會被
蓋住
的證明
我的美食地圖
看地圖
我的收藏
的證明
我的美食地圖
看地圖
我的收藏

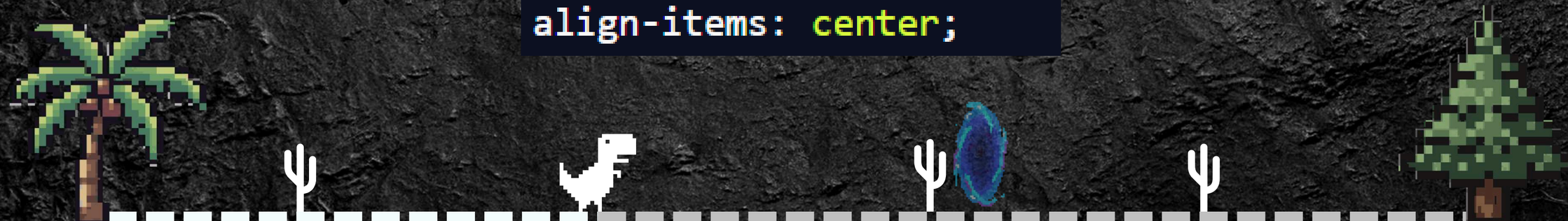
實作時間

10分鐘

重點整理:

- <details><summary>標題</summary>內容</details>
- .menu { position: sticky; }
- display:flex與 padding 要放在文字上
- .submenu { position: absolute; width: 100%; }

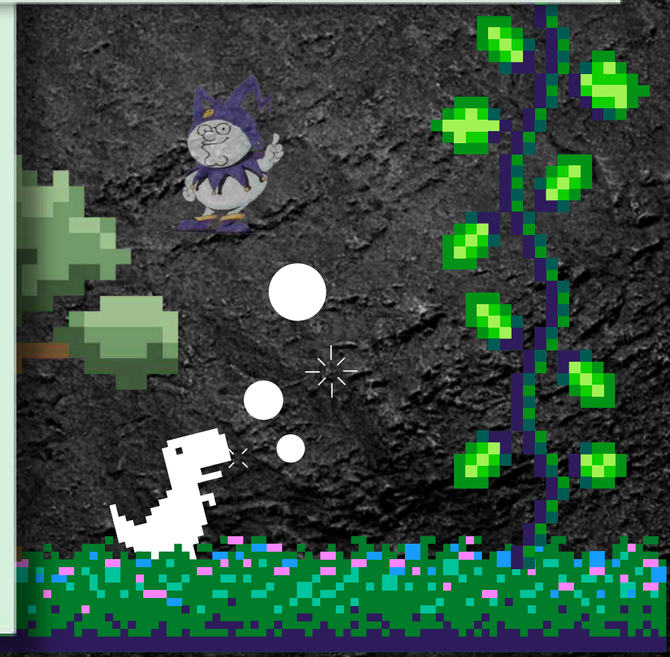
```
display: flex;
justify-content: center;
align-items: center;
```



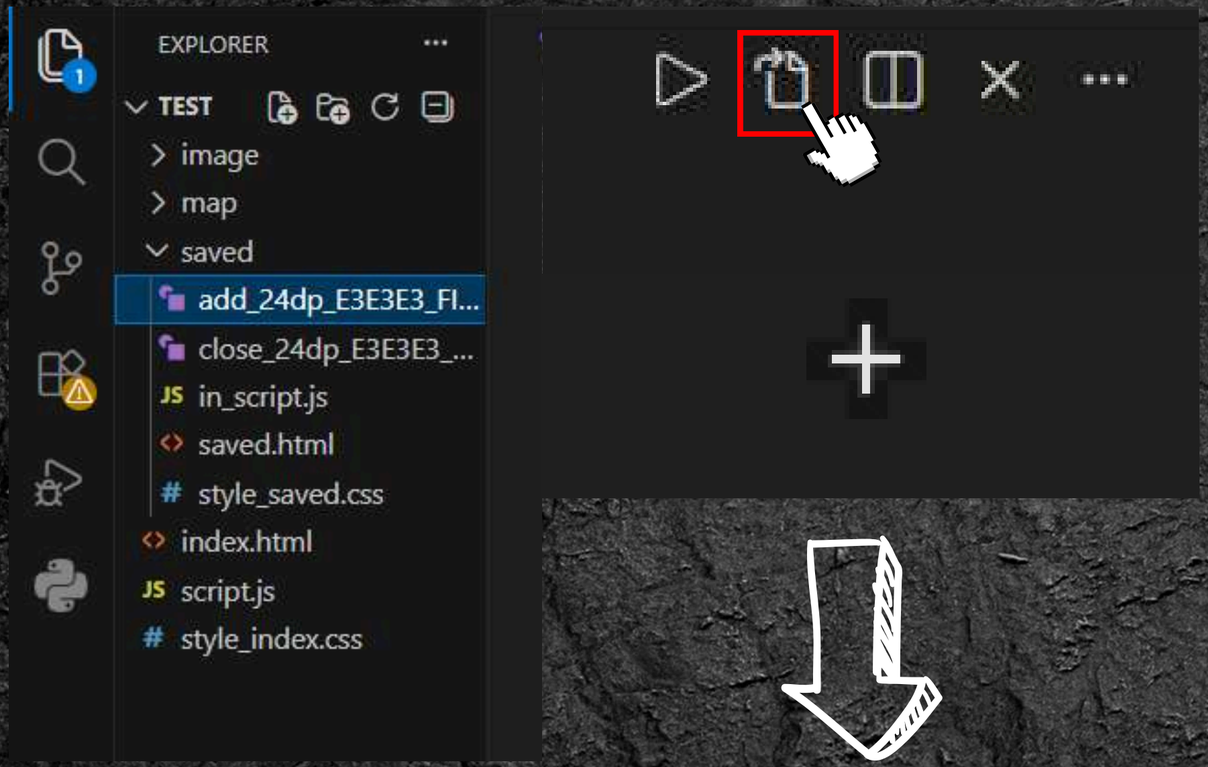
下課啦

（休息十分鐘）

Task03
橫向卡片



3-1. <svg>

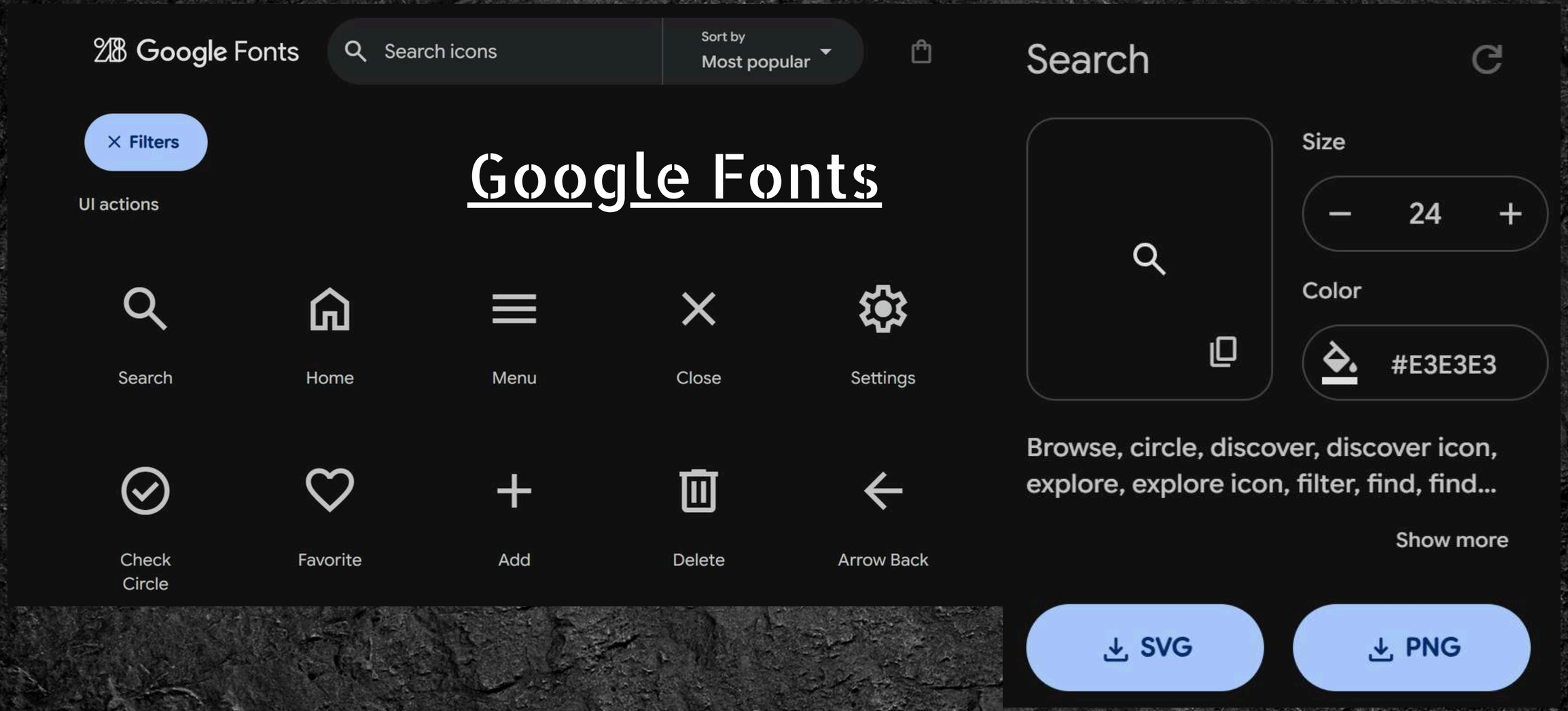



```
<label for="add" class="open-button">
    <svg xmlns="http://www.w3.org/2000/sv
</label>
```

```
<label for="add" class="open-button">

</label>
```

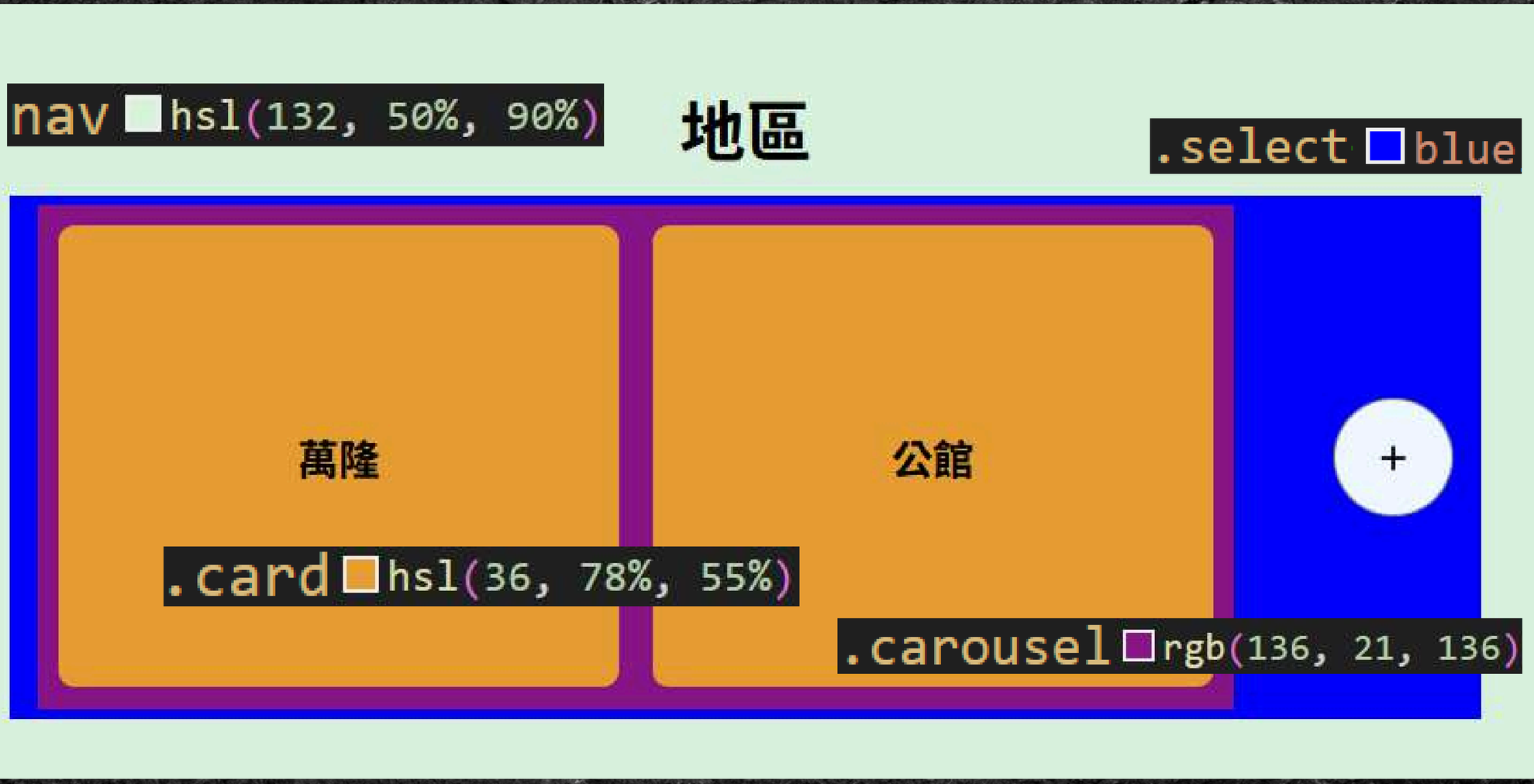
add_24dp_E3E3E3_FILL0_wght400_GRAD0_opsz24.svg saved\add_24dp_E3E3E3_FILL0_wght400_GRAD0_opsz24.svg

```
1   g/2000/svg" height="24px" viewBox="0 -960 960 960" width="24px" fill="#e3e3e3"><path d="M440-440H20
```



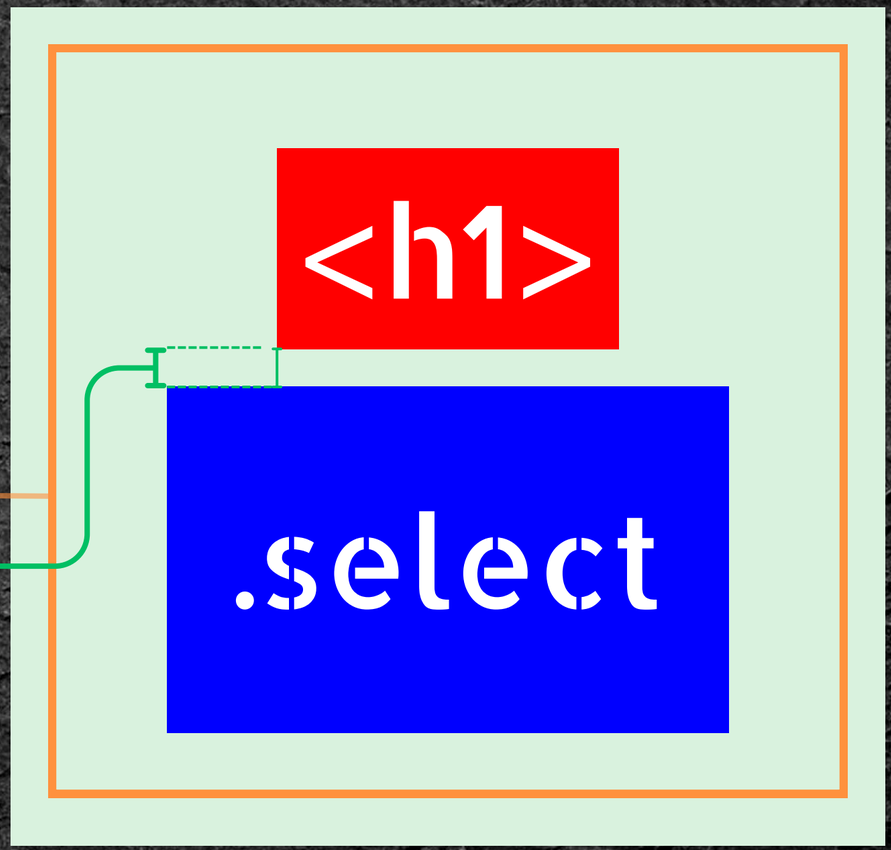
Google Fonts

3-2. 排版


nav hsl(132, 50%, 90%)
地區
.select blue
萬隆
公館
+
.card hsl(36, 78%, 55%)
.carousel rgb(136, 21, 136)

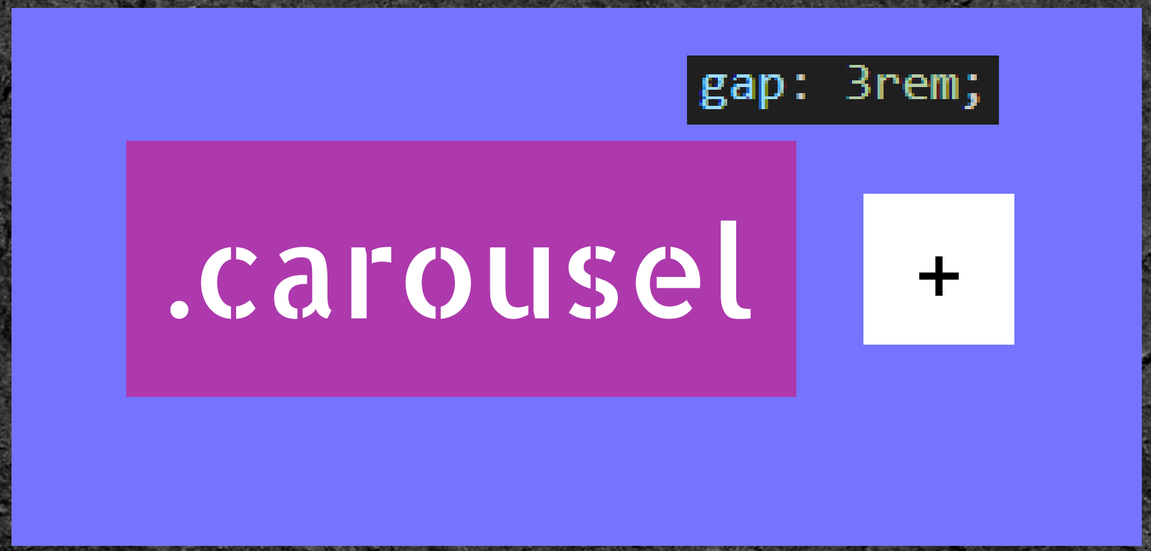
首先我想讓<nav>裡的物體縱向排列

```
display: flex;
flex-direction: column;
align-items: center;
justify-content: center;
height: 300px;
padding: 5%;
gap: 10px;
background-color: hsl(132, 5
```



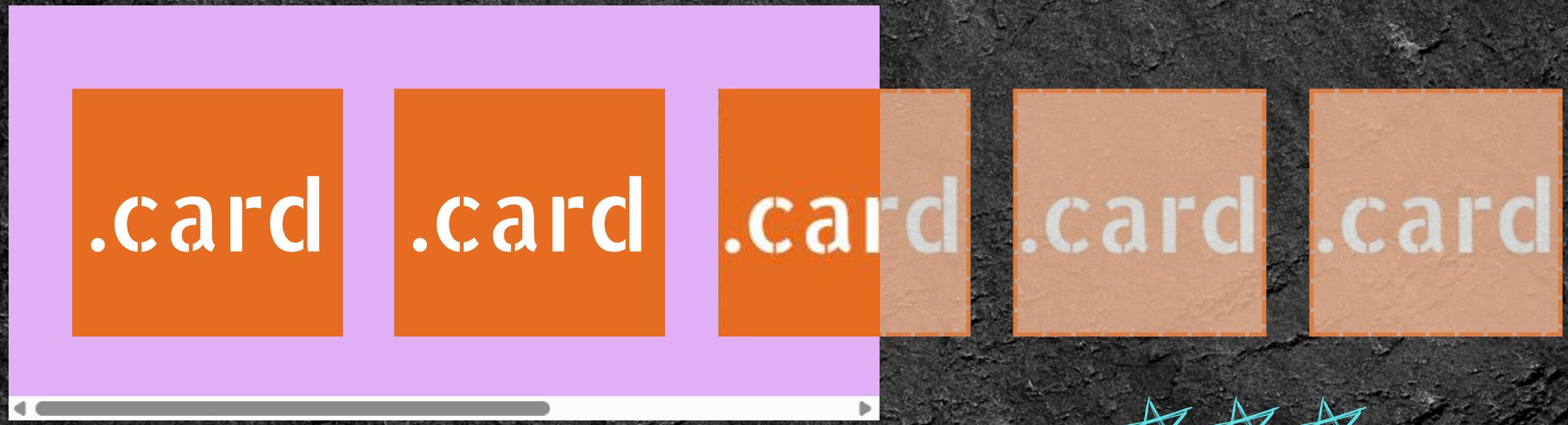
排列方向:縱向

接著把.select裡的物體橫向排列



最後是.carousel裡的所有卡片

.carousel




```
overflow-x: auto;
```

重要屬性: overflow (溢出)

決定內容物溢出視窗時怎麼辦

常用的overflow

- overflow
- overflow-y
- overflow-x

常用的屬性

- auto → 「若溢出」才顯示滾動軸
- hidden → 永遠隱藏
- scroll → 「隨時」顯示滾動軸
- visible → 即使溢出也永遠可見

3-3. 文字與<svg>調整

```
list-style: none;
font-size: 1.25rem;
font-weight: bold;
background-color: hsl(36, 78%, 55%);
border-radius: .5rem;
display: flex;
justify-content: center;
align-items: center;
```

3-3. 文字與<svg>調整

```
list-style: none;          列點樣式
font-size: 1.25rem;        文字大小
font-weight: bold;
background-color: hsl(36, 78%, 55%);
border-radius: .5rem;
display: flex;
justify-content: center;
align-items: center;
```

3-3. 文字與<svg>調整

```
list-style: none;          列點樣式
font-size: 1.25rem;        文字大小
font-weight: bold;         字體粗細
background-color: hsl(36, 78%, 55%);
border-radius: .5rem;
display: flex;
justify-content: center;
align-items: center;
```

3-3. 文字與<svg>調整

```
list-style: none;          列點樣式
font-size: 1.25rem;        文字大小
font-weight: bold;         字體粗細
background-color: hsl(36, 78%, 55%);   背景顏色
border-radius: .5rem;
display: flex;
justify-content: center;
align-items: center;
```

3-3. 文字與<svg>調整

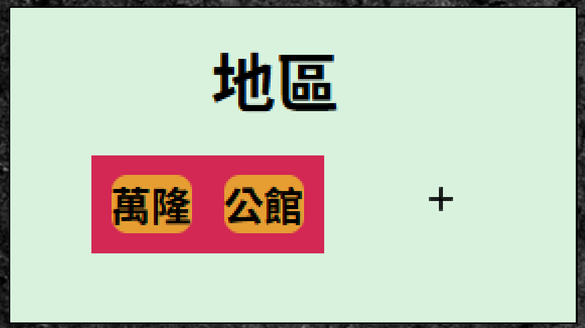
```
list-style: none;          列點樣式
font-size: 1.25rem;        文字大小
font-weight: bold;         字體粗細
background-color: hsl(36, 78%, 55%);   背景顏色
border-radius: .5rem;      邊角圓化
display: flex;
justify-content: center;
align-items: center;
```

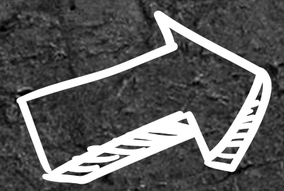
3-3. 文字與<svg>調整

```
list-style: none;          列點樣式
font-size: 1.25rem;        文字大小
font-weight: bold;         字體粗細
background-color: hsl(36, 78%, 55%);   背景顏色
border-radius: .5rem;      邊角圓化
display: flex;
justify-content: center;   主軸置中(x軸)
align-items: center;       副軸置中(y軸)
```

調整區塊大小①-把文字框撐開

當然也可以用padding



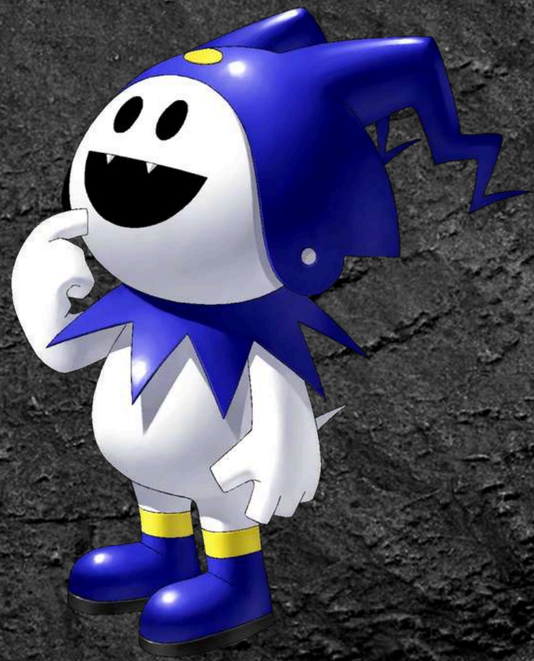



```
flex: 0 0 17rem; /*主軸尺寸(現在是x軸) 伸展0 壓縮0 基準17rem*/
height: 14rem; /*副軸尺寸(現在是y軸)*/
```

調整區塊大小② 直接給定方框大小

.open-button

```
height: 4rem;
width: 4rem;
border-radius: 50%;
border: 0.1px solid gray;
```



調整區塊大小② 直接給定方框大小

.open-button

```
height: 4rem;
width: 4rem;
border-radius: 50%;
border: 0.1px solid gray;
```

```
border-radius: 50%;
```

超過50%的效果等於50%

```
border: 0.1px solid gray;
```

寬度 樣式 顏色

solid	實線
dashed	虛線
dotted	點線

3-4. :hover時的效果

```
.card:hover{  右左位移   下上位移
    box-shadow: 0 0px 10px black;
    cursor: pointer;     模糊半徑
}
```

3-4. :hover時的效果

```css
.open-button svg{
    fill: black;
    width: 20px;
    height: auto;
    transform-origin: center;
}
```

```css
.open-button:hover{
    cursor: pointer;
    svg{
    transform: rotate(45deg) scale(1.5);
    fill: rgb(226, 148, 3);
    }
}
.open-button svg:hover {
    transform: rotate(45deg) scale(1.5);
    fill: rgb(226, 148, 3);
```

transform屬性:

改變外觀但不改變真實位置

```
transform-origin: center;
```
變化中心

transform屬性:

改變外觀但不改變真實位置

```
transform-origin: center;
```
變化中心

```
transform : rotate(45deg) + scale(1.5) + translate(50px, 20px) ... ;
```

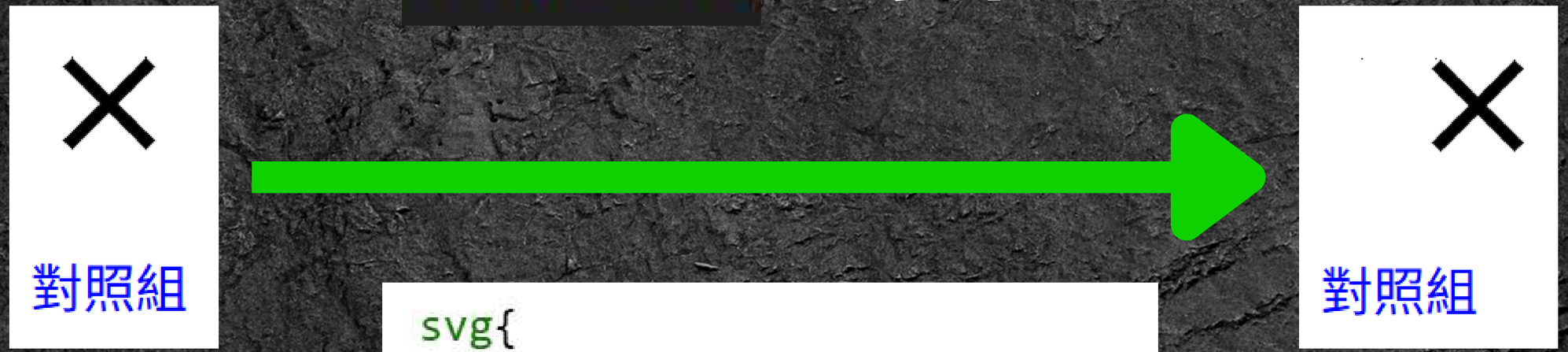
變化順序

順時針旋轉45度　放大1.5倍　向右移動50px 向下移動20px

```
transform: rotate(45deg) scale(1.5) translate(50px, 20px);
```

transform屬性:

translate 改變位置




```
svg{
  transform: translateX(50%);
}
```

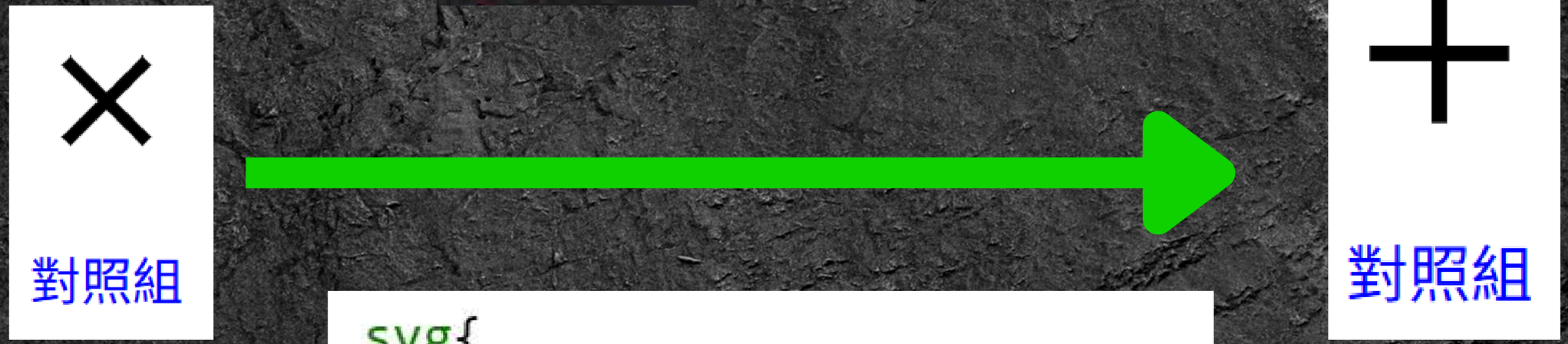
transform屬性:

`scale` 改變大小


```
svg{
  transform: scale(2);
}
```

transform屬性:

rotate 改變大小




```
svg{
  transform: rotate(45deg);
}
```

實作時間

20分鐘

Task03

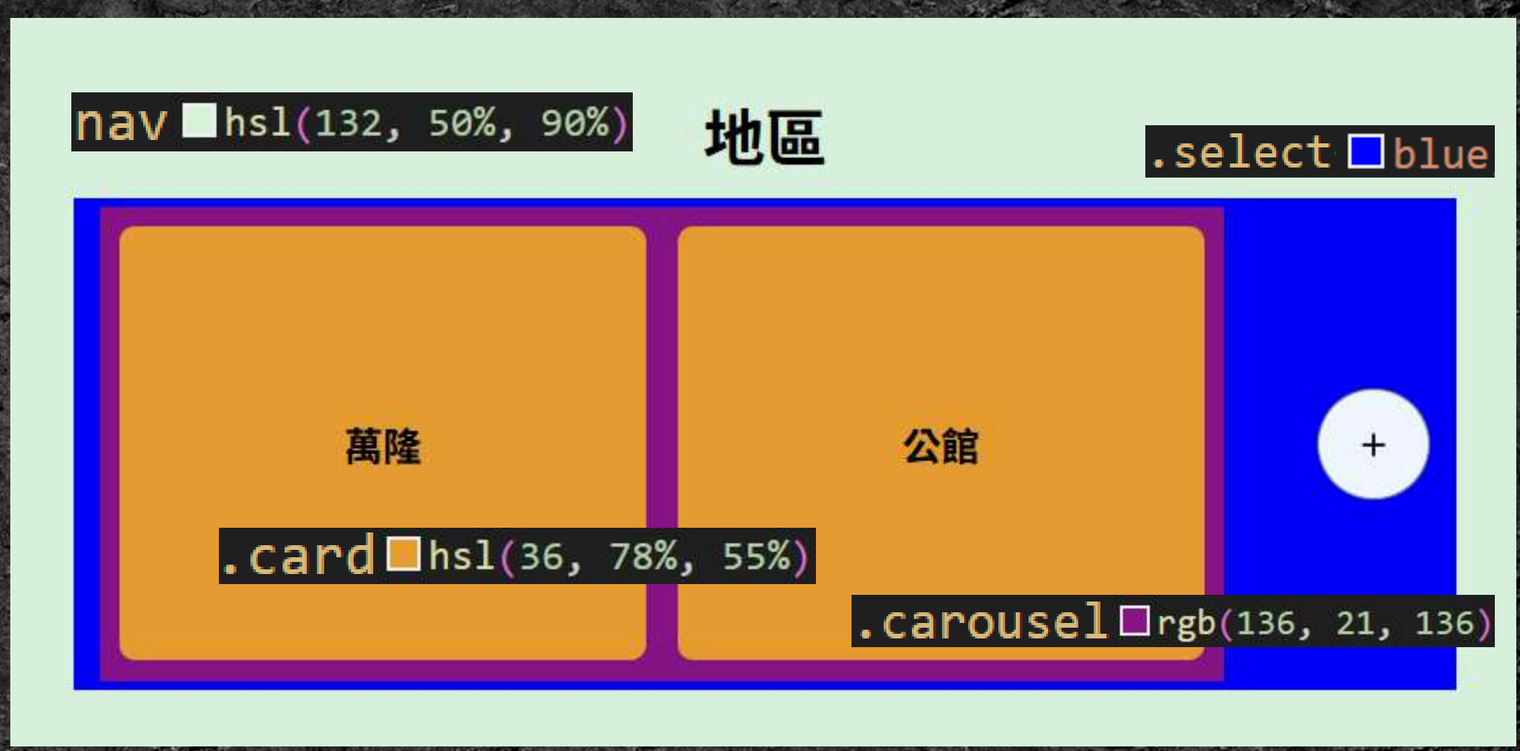
流程圖:

排版

- <nav>
- .select
- .carousel
- .card

樣式

- .open-button
- <svg>



Task~~05~~04
動畫與轉場

```
animation: name duration timing-function delay iteration-count direction fill-mode;
```

transition
transition-duration
transition-property
transition-timing-function
transition-delay
transition-behavior

animation-timing-function
animation-duration
animation-name
animation-delay
animation-iteration-count
animation-fill-mode
animation-direction
animation-play-state
animation-composition
animation-range

4-1. 轉場 transition

轉場是當表現形態改變時　用於平滑過度的屬性

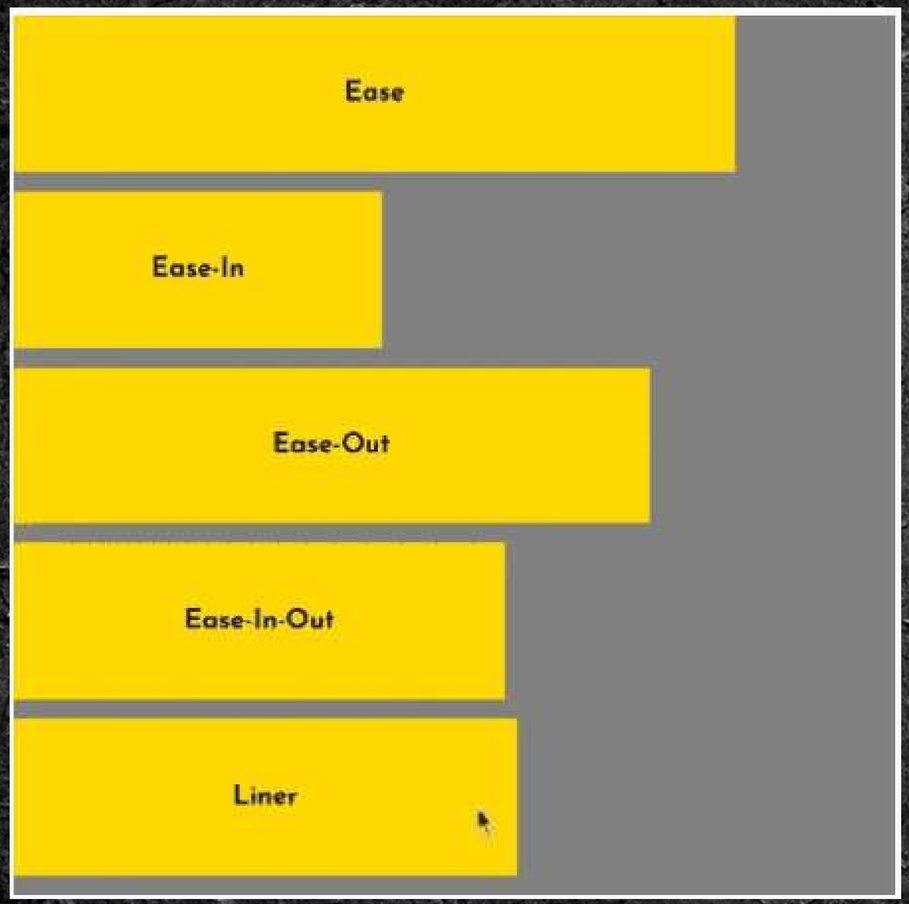
改變大小、位置、透明度

transition:　　屬性　　持續時間　　速度曲線　　延遲

transform
opacity 透明度
background-color

ease 預設，柔和
linear 等速
ease-in 慢 → 快
ease-out 快 → 慢
cubic-bezier 自訂

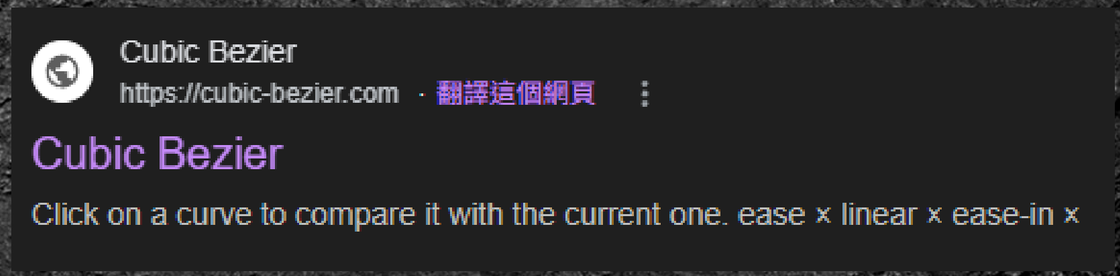
速度曲線展示



gif

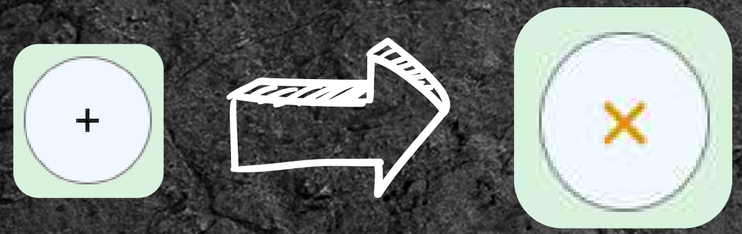
cubic-bezier

可直接找這個網站



轉場應用

把之前的加號按鈕拿出來用



```
.open-button svg{
    fill: black;
    width: 20px;
    height: auto;
    transform-origin: center;
    transition: transform 0.1s ease-in-out;
}
```

也可以給submenu加個彈跳

```
    transition: max-height 0.3s ease, opacity 0.2s ease;
}
.menu[open] .submenu {
    max-height: 100vh;
    opacity: 1;
}
```

4-2. 動畫 animation

相比於轉場，動畫是主動發生的

```
animation: name duration timing-function delay iteration-count direction fill-mode;
```

持續時間　速度曲線　延遲

決定重複次數

slide 滑行

bounce 彈跳

keyframe 自訂

alternate 正常

reverse-alternate 倒放

填both就好

@keyframe

想要自訂一個動畫就會需要keyframe

```
@keyframes pop-up{
    from{
        動畫開始時的參數
    }
    to{
        動畫結束時的參數
    }
}
```

```
@keyframes bounce {
    0% {
        動畫開始時的參數
    }
    75% {
        進行到75%時的參數
    }
    100% {
        動畫完成時的參數
    }
}
```

動畫範例-左側淡入

```
@keyframes left-fade-in{
    0%{
        opacity: 0%;
        transform: translateX(-20%);
    }
    100%{
        opacity: 100%;
        transform: translateX(0);
    }
}
.文字{
    font-
    width
    paddi
    displ
    flex-
    justify-content: center;
    align-items: center;
    animation: left-fade-in 0.5s ease-out 0.2s both ;
}
```

4-3. transition vs. animation

中間沒有點

transition	animation
被動（等狀態改變）	主動自動播放
hover	keyframe
單純	複雜

實作時間

15分鐘

100

重點整理:

```
animation: name duration timing-function delay iteration-count direction fill-mode;
```

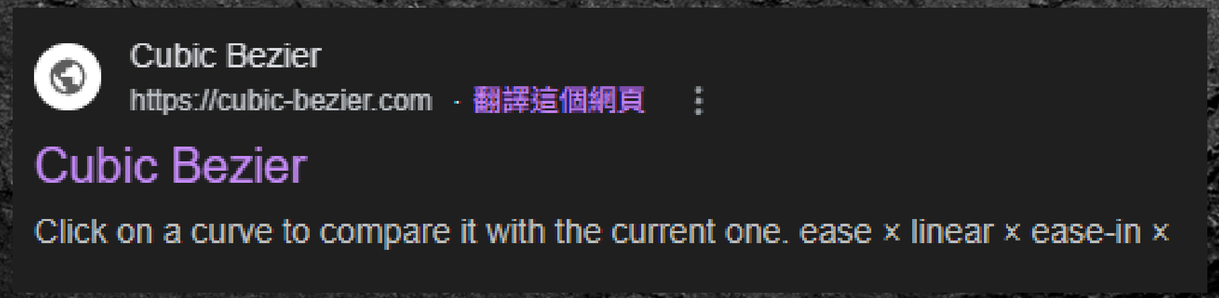
transition: 屬性 持續時間 速度曲線 延遲 次數 動畫方向

ease 預設，柔和

linear 等速

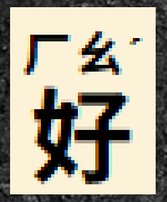
ease-in 慢 → 快

ease-out 快 → 慢



課堂總結

第一章:

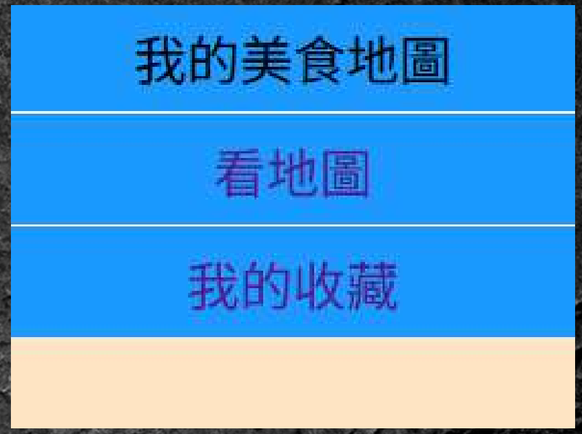


<ruby>
長度單位
font-文字屬性
text-文字呈現

早餐吃到飽

第二章:

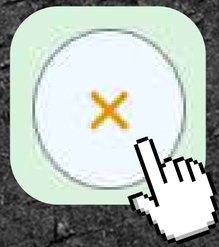
position
<details>
<summary>
製作導覽列



課堂總結

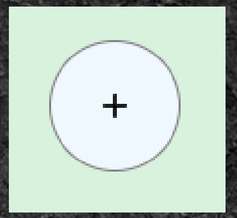
第三章:

- <svg>
- 排版
- overflow
- :hover時的效果



調整區塊大小的方法:

1. flex & height
2. width & height



第四章: 重點整理

```
animation: name duration timing-function delay iteration-count direction fill-mode;
```

transition: 屬性 持續時間 速度曲線 延遲 | 次數 動畫方向

transition	animation
被動（等狀態改變）	主動自動播放
hover	keyframe
單純	複雜

Task Pro

其他

▶ <abbr>

```
<abbr title="記住你終將死去">Memento Mori</abbr>
```

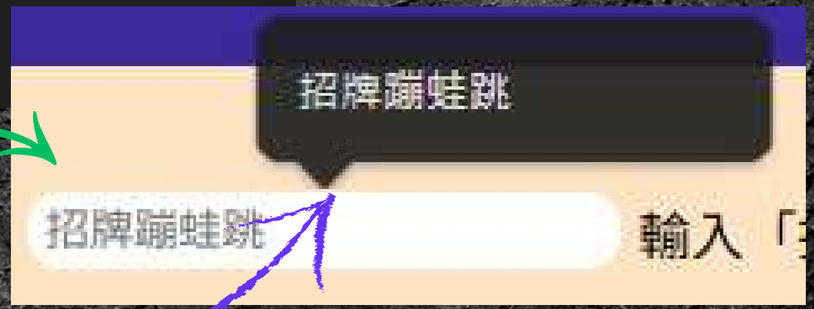
Memento Mori

記住你終將死去

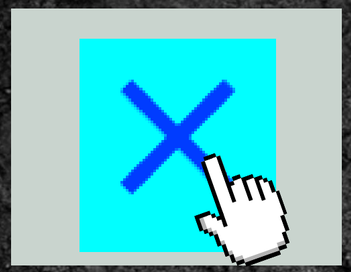
```
<input class="eat" placeholder="招牌蹦蛙跳" list="correct"/>
    <datalist id="correct">
        <option value="招牌蹦蛙跳"></option>
    </datalist>
```

▶ <input placeholder=””>

▶ <datalist> <option>



▶X符號的點擊區域判定問題



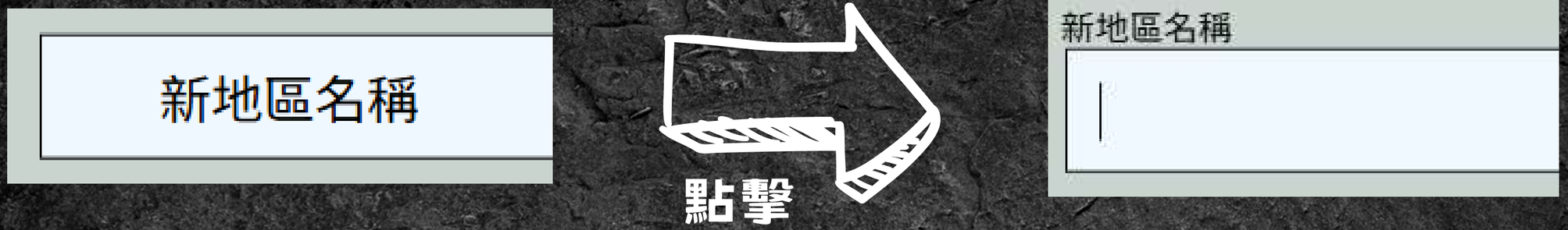
藍色這塊是label的區域

這次我只要svg自己能觸發checkedbox

解決方法:

```
.close-button svg{
    pointer-events: auto;
}
```

```
.close-button{
    pointer-events: none;
}
```

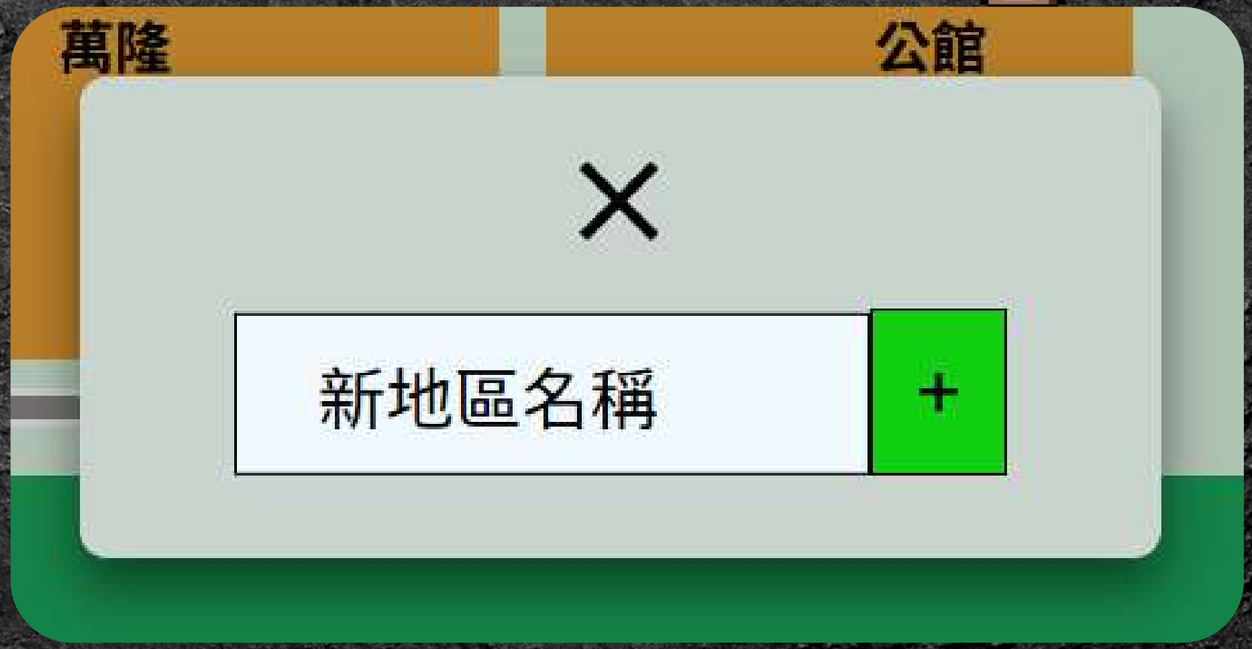


重點是 :focus 狀態

```
.content input:focus~label{
    top: 0;
    left: 2vw;
    font-size: 16px;
    background-color: transparent;
}
```

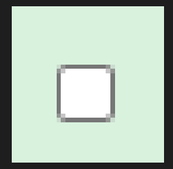
把absolte的文字根據
focus的狀態切換位置
中間再加上轉場

(原)Task04
彈出視窗



開始之前先把最上面的註解拿掉

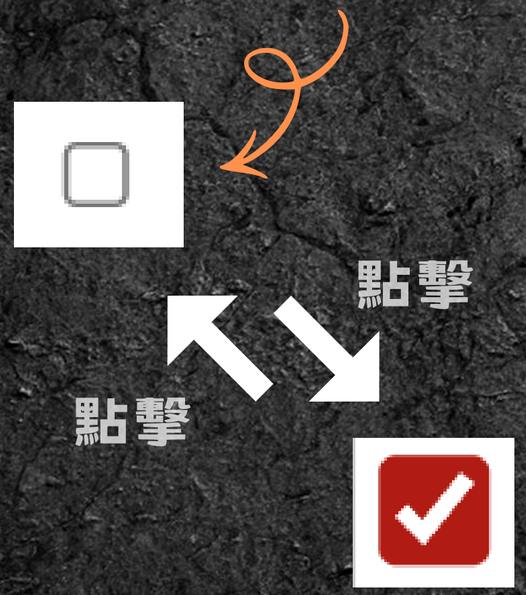
```
<input type="checkbox" id="add" >
<label for="add" class="open-button">
    <svg xmlns="http://www.w3.org/2000/svg" height="1rem" viewBox="0 -960 960 960" width=
</label>
<!--<label for="add" class="overlay"></label>
<div class="wrappedbox">
    <label for="add" class="close-button">
        <svg xmlns="http://www.w3.org/2000/svg" height="48px" viewBox="0 -960 960 960" w
    </label>
    <div class="content">
        <input class="new">
        <label for="">新地區名稱</label>
        <button>+</button>
    </div>
</div>-->
```



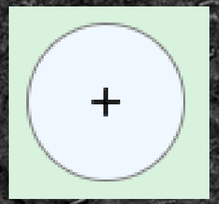
你會得到這個小東西

4-1. 概念總述

```
<input type="checkbox" id="add" >
```



這種彈出視窗本質上是
根據checkbox的狀態做切換

但我們希望用  和  控制checkbox

因此需要<label>將我想作用的區域包起來

```
<label for="add" class="open-button">
</label>

<label for="add" class="overlay"></label>
<label for="add" class="close-button">
</label>
```

因為寫了 for="add"

點擊這些label內的元素就同於

點擊checkbox

現在可以拿掉其他註解了

```
<input type="checkbox" id="add" >
<label for="add" class="open-button">
    <svg xmlns="http://www.w3.org/2000/svg" height="1rem" viewBox="0 -960 960 960" width
</label>
-------------------------------------------------------------------------------
<label for="add" class="overlay"></label>
<div class="wrappedbox">
    <label for="add" class="close-button">
        <svg xmlns="http://www.w3.org/2000/svg" height="48px" viewBox="0 -960 960 960" w
    </label>
    <div class="content">
        <input class="new">
        <button>+</button>
    </div>
</div>
```

4-2. 彈出視窗

先設定好彈出後的位置與狀態

```
.wrappedbox{
    position: fixed;
    display: flex;
    flex-direction: column;
    align-items: center;
    inset:40% 25%;
    height:max-content;
    width: 50%;
    background-color: rgb(201, 212, 206);
    padding: 20px;
    border-radius:10px;
    z-index: 20;
}
```

特別注意要用fixed

為了蓋過原本內容

由於checkbox狀態會改變...

```
~    class或其他  {
```

一般狀態(預設)

VS.

```
#add:checked ~  class或其他  {
```

改變後的狀態

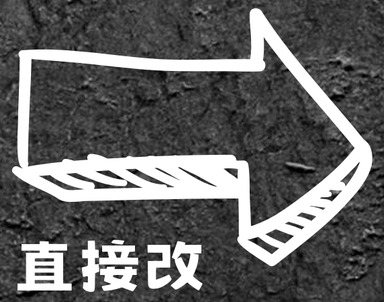
checkbox的id

我們要做的是在checkbox打勾時才出現的視窗

因此把上一頁設定好的內容

原封不動貼上去

```
.wrappedbox{
```




```
#add:checked ~.wrappedbox{
```

現在來調整checkbox不勾的狀態

預設就是沒有勾起來

```css
.wrappedbox{
    position: fixed;
    right: -100%;
    display: flex;
    flex-direction: column;
    align-items: center;
}
```

通常是看不到這個視窗的

所以用right:-100%把它推出螢幕

4-3. 遮罩

與彈出視窗的邏輯一樣 不過這次可以少寫一點東西

```css
.overlay{
    position: fixed;
    right: -100%;
}
#add:checked ~ .overlay{
    background-color: rgba(0,0,0, 0.2);
    height: 100%;
    width: 100%;
    inset: 0;
    z-index: 10;
}
```

為了「不」蓋過彈出視窗
選了一個比20小的值

```css
display: flex;
flex-direction: column;
align-items: center;
```

```css
padding: 20px;
```

The End

Thanks for listening!

▶ NTNU CSIE CAMP